# Stage 5 Step-by-Step Practical & Conceptual Recap: CRUD Verification

## 1. Architectural Synchronization

- **What You Did:** You synchronized your server.js route definitions with your openapi.json path schemas to resolve path mismatch (404) and syntax (400) errors.
- **Key Concept:** API documentation (Swagger) and the implementation code (Express) must be strictly mirrored. In a professional production environment, this is often automated via tools that generate OpenAPI specs directly from code annotations to prevent "documentation drift."
- **Sysadmin Analogy:** This is similar to ensuring a service configuration file (/etc/myapp.conf) matches the actual binary capability of the software. If the config calls for an endpoint that the code hasn't implemented, the system triggers an immediate routing failure.

## 2. Full CRUD Lifecycle Verification

You executed the complete CRUD lifecycle directly through the Swagger/OpenAPI interface, confirming the reliability of your backend:

### A. Create (POST /tasks)

- **Action:** Injected a new task ("Automate Kenyan SME invoices") into the system.
- **System Response:** 201 Created with a new, dynamic primary key (id: 4) assigned.
- **Significance:** Verified that the intake validation firewall is correctly parsing incoming JSON and assigning unique system identifiers.

### B. Read (GET /tasks/{id})

- **Action:** Querying the status of the newly created task via ID 4.
- **System Response:** 200 OK with the task object returned.
- **Significance:** Confirmed the lookup logic correctly traverses the memory array to return the specific record by its primary key.

### C. Update (PUT /tasks/{id})

- **Action:** Mutated the done status of task 4 to true.
- **System Response:** 200 OK.
- **Significance:** Validated the state-mutation logic. The server correctly identified the resource, applied the field update (maintaining data integrity), and returned the modified state.

### D. Delete (DELETE /tasks/{id})

- **Action:** Purged task 4 from the active memory store.
- **System Response:** 204 No Content.
- **Significance:** Confirmed that the deletion handler correctly performs silent purges, returning a clean HTTP status that signifies successful resource destruction.

# 3. The "Refresh" Protocol

- **Action:** Utilized Hard Refresh (Ctrl+F5) to clear browser-side caching of the openapi.json file.
- **Sysadmin Analogy:** Browser caches act as local client-side proxies. When updating API specifications, a hard flush is required to ensure the UI is reflecting the current backend state, identical to flushing a DNS cache on a local machine to resolve updated server records.